

```

1 import components.stack.Stack;
2
3 /**
4  * Controller class.
5  *
6  * @author Eric Zhou
7  */
8 public final class AppendUndoController1 implements AppendUndoController {
9
10     /**
11      * Model object.
12      */
13     private final AppendUndoModel model;
14
15     /**
16      * View object.
17      */
18     private final AppendUndoView view;
19
20     /**
21      * Updates view to display model.
22      *
23      * @param model
24      *         the model
25      * @param view
26      *         the view
27      */
28     private static void updateViewToMatchModel(AppendUndoModel model,
29         AppendUndoView view) {
30         /**
31          * Get model info
32          */
33         String input = model.input();
34         Stack<String> out = model.output();
35         Stack<String> temp = out.newInstance();
36         String output = "";
37         while (out.length() > 0) {
38             String s = out.pop();
39             temp.push(s);
40         }
41
42         while (temp.length() > 0) {
43             String s = temp.pop();
44             out.push(s);
45             output = output.concat(s);
46         }
47         /**
48          * Update view to reflect changes in model
49          */
50         view.updateInputDisplay(input);
51         view.updateOutputDisplay(output);
52     }
53
54     /**
55      * Constructor; connects {@code this} to the model and view it coordinates.
56      *
57      * @param model
58      *         model to connect to
59      * @param view
60      *         view to connect to
61      */
62     public AppendUndoController1(AppendUndoModel model, AppendUndoView view) {
63         this.model = model;
64         this.view = view;
65         /**

```

```

66A    /** Update view to reflect initial value of model
67        */
68    updateViewToMatchModel(this.model, this.view);
69    }
70
71    /**
72     * Processes reset event.
73     */
74    @Override
75    public void processResetEvent() {
76        /**
77         * Update model in response to this event
78         */
79        this.model.setInput("");
80        this.model.output().clear();
81        /**
82         * Update view to reflect changes in model
83         */
84        updateViewToMatchModel(this.model, this.view);
85    }
86
87    /**
88     * Processes copy event.
89     *
90     * @param input
91     *     value of input text (provided by view)
92     */
93    @Override
94    public void processAppendEvent(String input) {
95        /**
96         * Update model in response to this event
97         */
98        this.model.setInput("");
99        if (!input.equals("")) {
100            this.model.output().push(input);
101        }
102        /**
103         * Update view to reflect changes in model
104         */
105        updateViewToMatchModel(this.model, this.view);
106    }
107
108    @Override
109    public void processUndoEvent() {
110        Stack<String> out = this.model.output();
111        if (out.length() > 0) {
112            out.pop();
113        }
114        updateViewToMatchModel(this.model, this.view);
115    }
116
117 }
118

```